

RideSmart: Modelagem e Análise de Rotas Urbanas com Grafos na Cidade de Natal-RN

José Alex Araújo de Santana

Departamento de Engenharia da Computação e Automação (DCA)
Universidade Federal do Rio Grande do Norte, Natal, Brasil

Resumo—O projeto RideSmart modela o problema de seleção do melhor ponto de embarque em mobilidade urbana como caminho mínimo em grafos. Sobre a rede viária real de Natal-RN (18.576 nós, 48.180 arestas), obtida via OSMnx, foram implementados e comparados quatro algoritmos — Dijkstra Simples, Dijkstra com Heap, A* geográfico e Bellman-Ford — em cinco cenários com pesos de distância, tempo e trânsito sintético. O A* expandiu apenas 172 nós contra 773 do Dijkstra Simples, com tempo de 2,42 ms vs. 1.482,74 ms, confirmado por benchmark de 20 execuções (D.P. de 0,08 ms). O trânsito sintético elevou o custo temporal em 72,3% sem alterar a rota escolhida. Nenhum ponto de embarque alternativo superou a rota direta A→B, reflexo da boa conectividade local e da curta distância de 1,53 km entre origem e destino.

Index Terms—Grafos, Caminho Mínimo, Dijkstra, A*, Bellman-Ford, Mobilidade Urbana, OSMnx, Natal-RN

I. INTRODUÇÃO

Aplicativos de mobilidade como Uber e 99 selecionam rotas e pontos de embarque automaticamente, mas raramente expõem seus critérios ao usuário. O RideSmart investiga esse problema: dado um ponto de origem A , um destino B e um raio máximo de caminhada X , qual ponto de embarque P minimiza o custo total da viagem $A \rightarrow P$ (a pé) $+ P \rightarrow B$ (carro)?

A rede viária de Natal-RN foi escolhida por representar uma cidade de médio porte com malha heterogênea. Os objetivos são: (i) modelar a rede como dígrafo ponderado; (ii) implementar e comparar quatro algoritmos de caminho mínimo; (iii) analisar o impacto do trânsito sintético; e (iv) avaliar o trade-off entre caminhar até um ponto alternativo e partir diretamente de A .

II. MODELAGEM DO PROBLEMA

A rede viária é representada como dígrafo ponderado $G = (V, E, w)$: nós (V) são cruzamentos; arestas (E) são segmentos de via com sentido; pesos (w) codificam os critérios de otimização. O dígrafo é necessário para respeitar vias de mão única. Os dados foram obtidos via OSMnx [1], resultando em 18.576 nós e 48.180 arestas.

Três funções de custo foram definidas (Tabela I). O peso $\text{travel_time} = \text{length} / \text{speed_kph}$; o peso $\text{travel_time_traffic}$ multiplica travel_time por fatores por tipo de via (Tabela II), simulando hora de pico. Vias arteriais recebem fatores maiores (primary: $\times 2,0$; trunk: $\times 2,2$) e vias locais, menores (residential: $\times 1,1$; service: $\times 1,0$).

Tabela I
FUNÇÕES DE CUSTO E FATORES DE TRÂNSITO SINTÉTICO

Cenário	Peso	Unidade	Fator (ex.)
C1 — Distância	length	m	—
C2 — Tempo	travel_time	s	—
C3 — Trânsito	travel_time_traffic	s	$\times 1,0$ – $2,5$

Tabela II
FATORES DE CONGESTIONAMENTO POR TIPO DE VIA

Via	Fator	Via	Fator
motorway	$\times 2,5$	tertiary	$\times 1,4$
trunk	$\times 2,2$	residential	$\times 1,1$
primary	$\times 2,0$	unclassified	$\times 1,2$
secondary	$\times 1,7$	service / living	$\times 1,0$

Os pontos experimentais estão na Tabela III. A velocidade de caminhada foi fixada em 1,39 m/s (5 km/h) e o raio em $X = 500$ m, resultando em 3 candidatos P (Tabela IV).

Tabela III
PONTOS DE ORIGEM, DESTINO E CANDIDATOS P

Ponto	Local	Coord.
A (Origem)	UFRN — Campus Central	−5,8363, −35,1974
B (Destino)	Destino corrigido	−5,8416, −35,2102
Dist. linha reta A→B: 1.534 m Rota viária C4: 2.629 m		

Tabela IV
CANDIDATOS AO PONTO DE EMBARQUE P (≤ 500 m de A)

Nó (P)	Dist. A→P	T. Caminhada
1387156168	27 m	19 s
1387156245	39 m	28 s
1387156244	67 m	48 s

III. ALGORITMOS UTILIZADOS

Dijkstra Simples [2]: busca linear do nó de menor custo a cada iteração; complexidade $O(V^2)$. Simples de implementar, mas inviável em grafos grandes.

Dijkstra com Heap: substitui a busca linear por min-heap, reduzindo a complexidade para $O((V + E) \log V)$. Usa *lazy deletion*: entradas desatualizadas são ignoradas na extração.

A* Geográfico [3]: estende o Dijkstra com Heap pela função $f(n) = g(n) + h(n)$, onde $h(n)$ é a distância de Haversine entre n e B . Para pesos de tempo, $h(n)$ é dividida pela velocidade máxima (33,33 m/s), garantindo admissibilidade ($h(n) \leq h^*(n)$) e consistência, de modo que cada nó é expandido no máximo uma vez.

Bellman-Ford [4]: realiza $|V| - 1$ relaxamentos sobre todas as arestas; complexidade $O(V \cdot E)$. Suporta arestas de peso negativo e detecta ciclos negativos — vantagens irrelevantes em redes viárias, mas relevantes como algoritmo de referência da literatura.

Tabela V
COMPLEXIDADE DOS ALGORITMOS

Algoritmo	Complexidade	Estrutura
Dijkstra Simples	$O(V^2)$	Lista linear
Dijkstra Heap	$O((V + E) \log V)$	Min-heap
A*	$O(E \log V)$	Min-heap + heurística
Bellman-Ford	$O(V \cdot E)$	Relaxamento iterativo

IV. EXPERIMENTOS E RESULTADOS

Os experimentos cobriram cinco cenários: C1 (distância), C2 (tempo), C3 (trânsito), C4 (rota direta A→B, caso-base) e C5 (ganho ao caminhar). Para C1–C3, cada candidato P foi avaliado com os quatro algoritmos; para C4, apenas a rota direta. A Tabela VI compara o desempenho no cenário C4.

Tabela VI
CENÁRIO C4 — ROTA DIRETA A→B (PESO: DISTÂNCIA)

Algoritmo	Custo (m)	Nós Exp.	Tempo (ms)
Dijkstra Simples	2.629	773	1.482,74
Dijkstra Heap	2.629	774	5,27
A*	2.629	172	2,42
Bellman-Ford	2.629	116	4.667,20

Todos convergiram para 2.629 m, confirmando a correção das implementações. Uma única medição de tempo é suscetível a variações do sistema operacional (escalonamento, cache, I/O). Para verificar se os resultados da Tabela VI são reproduzíveis, cada algoritmo foi executado 20 vezes consecutivas no mesmo cenário C4 e as estatísticas descritivas foram registradas na Tabela VII. Desvios padrão pequenos indicam algoritmos estáveis; desvios grandes revelam sensibilidade ao estado do sistema, o que compromete a previsibilidade em uso real.

A Tabela VIII resume o melhor P por cenário e a Tabela IX o ganho ao caminhar.

Tabela VII
BENCHMARK DE ROBUSTEZ — 20 EXECUÇÕES (CENÁRIO C4)

Algoritmo	Média (ms)	D.P. (ms)	Med. (ms)	Mín. (ms)	Máx. (ms)
Dijkstra Simples	1.520,01	142,82	1.461,69	1.411,11	1.993,78
Dijkstra Heap	4,68	0,34	4,66	4,21	5,92
A*	2,01	0,08	1,98	1,94	2,21
Bellman-Ford	4.518,05	472,34	4.610,60	3.727,78	5.414,64

Tabela VIII
MELHOR P POR CENÁRIO (DIJKSTRA HEAP)

Cenário	P Ótimo	Caminhada	Custo
C1 — Distância	1387156245	39 m	2.589,3 m
C2 — Tempo	1387156245	39 m	208,1 s
C3 — Trânsito	1387156245	39 m	358,5 s

V. DISCUSSÃO CRÍTICA

Eficiência computacional. O A* expandiu 77,7% menos nós que o Dijkstra Simples (172 vs. 773) e foi 613× mais rápido (2,42 ms vs. 1.482,74 ms), com D.P. de apenas 0,08 ms — evidência de comportamento estável, reflexo da heurística geográfica. O Dijkstra Heap, com 5,27 ms e D.P. de 0,34 ms, demonstra que a estrutura de dados é o fator dominante sobre o Dijkstra Simples (D.P. de 142,82 ms), cuja instabilidade cresce com o espaço explorado. O Bellman-Ford, apesar de expandir apenas 116 “nós” (iterações de relaxamento), consumiu 4.667,20 ms — sua complexidade $O(V \cdot E)$ obriga percorrer todas as 48.180 arestas repetidamente.

Distância vs. tempo. Os três cenários elegeram o mesmo P ótimo (1387156245), mas os custos diferem substancialmente: 2.589,3 m (C1), 208,1 s (C2) e 358,5 s (C3). Isso evidencia que menor distância não implica menor tempo, pois o tipo e a velocidade da via são determinantes.

Trânsito sintético. O custo temporal do melhor P saltou de 208,1 s (C2) para 358,5 s (C3), aumento de 72,3%, reflexo do fator $\times 2,0$ nas vias *primary* predominantes na rota. A rota selecionada permaneceu a mesma, mas o impacto temporal demonstra que a modelagem de congestionamento é crítica para a experiência do usuário.

Trade-off caminhar / tempo e quando caminhar atrapalha. Todos os candidatos P resultaram em ganho negativo (Tabela IX): o melhor caso perdeu apenas 23 s. Caminhar atrapalha em três condições identificadas neste experimento:

Tabela IX
CENÁRIO C5 — GANHO AO CAMINHAR ATÉ P (BASE: 363,3 s)

P	Walk	T. Walk	T. Carro	Ganho
1387156168	27 m	19 s	508 s	−164 s
1387156245	39 m	28 s	358 s	−23 s
1387156244	67 m	48 s	363 s	−48 s

(i) quando a origem já está bem inserida na malha — o Campus da UFRN possui acesso direto a vias secundárias e terciárias; (ii) quando a distância $A \rightarrow B$ é curta (1,53 km), tornando qualquer acréscimo de caminhada uma fração elevada do custo total; e (iii) quando nenhum candidato P oferece acesso privilegiado a vias mais rápidas do que as acessíveis diretamente de A. Em cenários com distâncias maiores ou origens em áreas de malha precária, caminhar até um ponto com melhor conectividade poderia gerar ganho positivo.

A* expandiu menos nós que o Dijkstra? Sim. A heurística geográfica direciona a busca para a região do destino, eliminando grande parte das explorações desnecessárias. O Dijkstra (ambas as versões) explora o grafo de forma radial, sem informação direcional.

Limitações. O filtro de candidatos P usa distância Haversine (linha reta), podendo incluir pontos fisicamente inacessíveis. O trânsito sintético é estático, sem variação por hora do dia. O grafo não modela tempo de espera em semáforos, relevante em vias *primary*. Com apenas 3 candidatos P, o espaço de busca é limitado.

VI. CONCLUSÃO

O RideSmart demonstrou a viabilidade de modelar a seleção de ponto de embarque como caminho mínimo em dígrafo ponderado real. O A* superou os demais em eficiência (2,42 ms, D.P. 0,08 ms); o Dijkstra Heap oferece equilíbrio prático (5,27 ms); o Bellman-Ford, embora correto, é inadequado para grafos viários positivos em produção. O trânsito sintético impactou 72,3% o custo temporal sem mudar a rota; caminhar não gerou ganho neste cenário específico.

Para aproximar o modelo de um aplicativo real, destacamos: (i) dados reais de tráfego via Google Maps ou HERE com variação temporal por hora e dia da semana; (ii) *Contraction Hierarchies* [5] para escalabilidade em produção; e (iii) suporte multimodal combinando caminhada, ônibus e bicicleta em custo unificado.

REFERÊNCIAS

- [1] G. Boeing, “OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks,” *Comput. Environ. Urban Syst.*, vol. 65, pp. 126–139, 2017.
- [2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numer. Math.*, vol. 1, pp. 269–271, 1959.
- [3] P. E. Hart, N. J. Nilsson e B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.
- [4] R. Bellman, “On a routing problem,” *Q. Appl. Math.*, vol. 16, pp. 87–90, 1958.
- [5] T. H. Cormen *et al.*, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.